

SymPy Bug Report

1 Bug 63: Fourier Transform Has Wrong Sign at Negative Frequencies

1.1 Minimal Reproducer

```
from sympy import *

x, k = symbols("x k", real=True)
expr = cos(x) / (x**2 + 1)
F = fourier_transform(expr, x, k)

value = Rational(-1, 3)
expected = pi * (exp(-Abs(2*pi*value - 1)) + exp(-Abs(2*pi*value + 1))) / 2

print("transform:", F)
print("SymPy at k=-1/3:", N(F.subs(k, value), 50))
print("shift-theorem expected:", N(expected, 50))
print("difference:", N(F.subs(k, value) - expected, 50))
```

1.2 Observed Behavior

```
SymPy at k=-1/3:
-0.59697240701635906558464748995740185380690547910845
+ 2.2927016814328156775234560803181663759131427061003e-64*I

shift-theorem expected:
0.59697240701635906558464748995740185380690547910845

difference:
-1.1939448140327181311692949799148037076138109582169
+ 2.2927016814328156775234560803181663759131427061003e-64*I
```

SymPy returns the negative of the expected real value at $k = -1/3$, up to a negligible numerical imaginary roundoff term. This is not merely an alternate closed form: the sign of the Fourier transform value is wrong at this negative frequency.

1.3 Expected Behavior

Under SymPy's Fourier transform convention

$$\mathcal{F}(f)(k) = \int_{-\infty}^{\infty} f(x) e^{-2\pi i x k} dx,$$

the transform should be

$$\text{fourier_transform}\left(\frac{\cos x}{x^2 + 1}, x, k\right) = \frac{\pi}{2} \left(e^{-|2\pi k - 1|} + e^{-|2\pi k + 1|} \right)$$

for real k . In particular, at $k = -1/3$ the value should be the positive number

$$\frac{\pi}{2} \left(e^{-|-2\pi/3 - 1|} + e^{-|-2\pi/3 + 1|} \right) \approx 0.59697240701635906558464748995740185380690547910845.$$

1.4 Mathematical Explanation

The standard transform for the Cauchy kernel under SymPy's convention is

$$\mathcal{F}\left(\frac{1}{x^2 + 1}\right)(k) = \pi e^{-2\pi|k|}.$$

Since

$$\cos x = \frac{e^{ix} + e^{-ix}}{2},$$

the modulation/shift theorem gives

$$\begin{aligned} \mathcal{F}\left(\frac{\cos x}{x^2 + 1}\right)(k) &= \frac{1}{2} \mathcal{F}\left(\frac{e^{ix}}{x^2 + 1}\right)(k) + \frac{1}{2} \mathcal{F}\left(\frac{e^{-ix}}{x^2 + 1}\right)(k) \\ &= \frac{\pi}{2} \left(e^{-2\pi|k - 1/(2\pi)|} + e^{-2\pi|k + 1/(2\pi)|} \right) \\ &= \frac{\pi}{2} \left(e^{-|2\pi k - 1|} + e^{-|2\pi k + 1|} \right). \end{aligned}$$

This expression is positive and even in k , as also follows directly from the fact that $\cos(x)/(x^2 + 1)$ is real and even. SymPy's negative value at $k = -1/3$ violates both the shift-theorem formula and the required evenness, so it is not a harmless branch representation or simplification choice.

1.5 Source-Level Diagnosis

The diagnosis narrows the issue to the Meijer definite integration path in `sympy/integrals/meijerint.py`, especially around `sympy/integrals/meijerint.py:1834` and `sympy/integrals/meijerint.py:2048`. The public call path is:

```
fourier_transform(f, x, k)
-> FourierTransform(f, x, k).doit()
-> FourierTypeTransform._compute_transform
-> _fourier_transform(f, x, k, 1, -2*pi, "Fourier")
```

The last step forms the definite integral

$$\int_{-\infty}^{\infty} \frac{\cos x}{x^2 + 1} e^{-2\pi i x k} dx.$$

That integral is evaluated through `meijerint_definite`. The real-line integral is split at zero, the two half-line integrals are passed through `_meijerint_definite_2` and `_meijerint_definite_4`, and the result is assembled through the two-Meijer- G rewrite/integration path.

The diagnosis identifies the following source-level pattern as relevant:

```

# meijerint_definite, -oo to oo case
res1 = _meijerint_definite_2(f.subs(x, x + c), x)
res2 = _meijerint_definite_2(f.subs(x, c - x), x)
res = res1 + res2

# _meijerint_definite_4
gs = _rewrite2(f, x)
...
res += fac*_int0oo(f1, f2, x)

# _int0oo
eta, _ = _get_coeff_exp(g1.argument, x)
omega, _ = _get_coeff_exp(g2.argument, x)
return meijerg(a1, a2, b1, b2, omega/eta)/eta

```

The generated half-line formulas contain branch-sensitive factors involving `polar_lift(k)**2`, `Abs(k)`, and an explicit denominator `k`. After the transform simplification in `sympy/integrals/transforms.py` applies `piecewise_fold`, `powdenest(..., polar=True)`, and `simplify`, the public transform retains an overall `Abs(k)/k` sign factor. This makes the result odd under $k \mapsto -k$ even though the input function is real and even and the correct transform is even.

The diagnosis does not isolate a single faulty line inside the Meijer machinery, but it reports that the bad sign is already present before `_fourier_transform` returns. A plausible fix direction is to preserve the real negative-frequency branch information in the Meijer half-line formulas, or to return a conditional, `Piecewise`, or absolute-value expression equivalent to the shift-theorem result instead of a formula with an `Abs(k)/k` factor.

1.6 Additional Failing Instances

The same sign error appears across the parametrized negative-frequency family checked independently. Each case evaluates the same transform

$$\frac{\pi}{2} \left(e^{-|2\pi k-1|} + e^{-|2\pi k+1|} \right)$$

at a negative rational frequency. The expected value is positive in every case because it is a sum of positive exponentials; SymPy instead returns the negative of that value, again with only negligible numerical imaginary roundoff.

Input / Expression	SymPy behavior	Expected behavior
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-1/3)$	-0.596972407016359	0.596972407016359
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-1/4)$	-1.00774422210450	1.00774422210450
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-1/2)$	-0.209489441961013	0.209489441961013
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-2/3)$	-0.0735139945788715	0.0735139945788715
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-1)$	-0.00905285962202646	0.00905285962202646
$\mathcal{F}\{\cos x/(x^2 + 1)\}(-3/2)$	-0.000391209535759656	0.000391209535759656

1.7 Related Issues Assessment

The initial deduplication check did not surface a public match for this exact integrand. A follow-up audit identified an open public issue that the original search missed: [sympy/sympy#25137](#), “In-

tegrate returns wrong answer for fourier transform of sinc” (open; labels `integrals.transforms`, `integrals.meijerg`). That issue documents the same meijerg Fourier-transform abs/k sign family: with variables declared `real=True`, the meijerg path returns a wrong result carrying a spurious $\text{Abs}(k)/k$ -type factor (correct when not real), and a maintainer comment gives the matching root cause, namely a branch-dependent $\text{sqrt}(1/y^{**2})$ collapsing to $\text{sign}(y) = \text{abs}(y)/y$. The earlier “no public precedent” framing is therefore overstated: this is a known failure family. The specific integrand $\cos x/(x^2 + 1)$ and its negative-frequency sign error are plausibly a new instance of that family, so this concrete reproducible case remains individually actionable as a bug report and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Abs, N, Rational, cos, exp, fourier_transform, pi, symbols

x, k = symbols("x k", real=True)
TRANSFORM = fourier_transform(cos(x) / (x**2 + 1), x, k)

@pytest.mark.parametrize(
    "value",
    [
        Rational(-1, 3),
        Rational(-1, 4),
        Rational(-1, 2),
        Rational(-2, 3),
        Rational(-1, 1),
        Rational(-3, 2),
    ],
)
def test_fourier_transform_shifted_cauchy_kernel_negative_frequencies(value):
    expected = pi * (exp(-Abs(2*pi*value - 1)) + exp(-Abs(2*pi*value + 1))) / 2
    assert abs(complex(N(TRANSFORM.subs(k, value) - expected, 80))) < 1e-40
```